Contract Testing Business Demo

Complete Architecture and Flow Guide

Repository: Avinash258/contract-testing-business-demo

Prepared for: QA / Test Architecture learners

Document date: 27 October 2026

Source: docs-analysis/00 … 06 Markdown pack (explain-only analysis)

Diagrams: 19 Mermaid figures rendered to PNG

This document merges the docs-analysis Markdown files without rewriting their content. Mermaid diagrams are embedded as figures. Commands described in the source were not executed for the analysis.

Table of Contents

Table of Contents 2
Document outline (static) 4
List of Diagrams 4
00 — Index and Discovery 5
TaxMesh Contract-Testing Demo — Analysis Index 5
How to read these files 6
One-line verdict 8
Phase 1 discovery table (full) 8
Repo tree + purpose 9
Phase gate checklist 9
Related upstream docs (already in repo) 9
01 — Beginner Concepts 10
01 — Beginner Concepts (mapped to THIS repo) 10
1. What is contract testing? 10
2. Why it exists (vs Integration vs E2E) 11
3. Core terms — only what THIS repo uses 11
4. Comparison table (memorize this) 12
5. Business problem THIS demo solves 12
6. Mini glossary of THIS project's "contract" 14
Gate 2 14
02 — Architecture 15
02 — Architecture 15
1. System context (C4-style) 16
2. Component diagram 18
3. Tech stack table 18
4. Dependency table (module → depends on → why) 19
5. Data / contract flow 19
Contract mesh (downstream solid / upstream dashed) 20
Upload-driven filing flow (separate from CT runner) 20
6. Class / module diagram (logical) 21
Gate 3 21
03 — File-by-File Guide 22
03 — File-by-File Guide 22
Priority legend 22
1. Master inventory table 22
Root / config 22
Contracts 22
Shared runtime 23

Gateway / frontend 23
Services (entry points) 23
Scripts 24
Testdata / Postman / Docs 24
2. Deep dive — 8 most important files (block by block) 24
A. `shared/contracts.js` — Must understand 24
B. `shared/payloads.js` — Must understand 25
C. `contracts/run-contracts.js` — Must understand 25
D. `shared/createService.js` — Must understand 25
E. `scripts/start-all.js` — Must understand 25
F. `gateway/index.js` — Must understand 26
G. `gateway/filingRoutes.js` — Should know / Must for demo 26
H. `shared/filingEngine.js` — Should know 26
3. Quick “read order” for beginners 27
Gate 4 29
04 — End-to-End Flow 30
04 — End-to-End Flow 30
1. Lifecycle sequence (how contracts work HERE) 30
2. Numbered lifecycle table 30
3. Happy path (contracts stay valid) 31
4. Contract broken path (theoretical) 33
5. Sample contract explained — CT-02 35
6. Full contract matrix (evidence) 36
7. Filing happy path (business demo, not the CT runner) 37
Gate 5 37
05 — How It Runs (Explained) 38
05 — How It Runs (Explained Only — NOT Executed) 38
1. Prerequisites 38
2. Commands table 38
What `npm start` boots (`scripts/start-all.js`) 39
What `test:contracts` does inside `check(ct)` 40
3. Expected PASS vs FAIL (not real output) 42
PASS — expected behaviour 42
FAIL — expected behaviour 42
Filing API FAIL (separate) 42
4. Useful URLs (when running — described from README) 42
5. Troubleshooting 42
6. Docker layout (explained) 43
Gate 6 43

06 — Glossary and Q&A 44
06 — Glossary + Q&A 44
1. Glossary 44
2. Interview-style Q&A (15) — answers from THIS project 45
3. “Explain this project in 2 minutes” script 45
4. Learning path 46
5. Final summary of uncertainties 46
Gate 7 46
Build metadata 48

Document outline (static)

1. **Index and Discovery** — source section “Index and Discovery”
2. **Beginner Concepts** — source section “Beginner Concepts”
3. **Architecture** — source section “Architecture”
4. **File-by-File Guide** — source section “File-by-File Guide”
5. **End-to-End Flow** — source section “End-to-End Flow”
6. **How It Runs (Explained)** — source section “How It Runs (Explained)”
7. **Glossary and Q&A** — source section “Glossary and Q&A”

List of Diagrams

8. **Figure 1:** How to read these analysis files _(source: section “Index and Discovery”, type: flowchart)_
9. **Figure 2:** With vs without contracts _(source: section “Beginner Concepts”, type: flowchart)_
10. **Figure 3:** Test pyramid speed layers _(source: section “Beginner Concepts”, type: flowchart)_
11. **Figure 4:** Downstream and upstream example _(source: section “Beginner Concepts”, type: flowchart)_
12. **Figure 5:** Business problem this demo solves _(source: section “Beginner Concepts”, type: flowchart)_
13. **Figure 6:** System context (C4-style) _(source: section “Architecture”, type: C4Context)_
14. **Figure 7:** Gateway to six microservices _(source: section “Architecture”, type: flowchart)_
15. **Figure 8:** Component diagram _(source: section “Architecture”, type: flowchart)_
16. **Figure 9:** Contract data flow sequence _(source: section “Architecture”, type: sequenceDiagram)_
17. **Figure 10:** Contract mesh CT-00 to CT-17 _(source: section “Architecture”, type: flowchart)_
18. **Figure 11:** Upload-driven filing flow _(source: section “Architecture”, type: flowchart)_
19. **Figure 12:** Module class diagram _(source: section “Architecture”, type: classDiagram)_
20. **Figure 13:** Recommended beginner read order _(source: section “File-by-File Guide”, type: flowchart)_
21. **Figure 14:** Contract lifecycle sequence _(source: section “End-to-End Flow”, type: sequenceDiagram)_
22. **Figure 15:** Happy path verification _(source: section “End-to-End Flow”, type: flowchart)_
23. **Figure 16:** Contract broken path _(source: section “End-to-End Flow”, type: flowchart)_
24. **Figure 17:** Filing happy path sequence _(source: section “End-to-End Flow”, type: sequenceDiagram)_
25. **Figure 18:** test:contracts check flow _(source: section “How It Runs (Explained)”, type: flowchart)_
26. **Figure 19:** Learning path progression _(source: section “Glossary and Q&A”, type: flowchart)_

00 — Index and Discovery

Source file: 00-README-INDEX.md

TaxMesh Contract-Testing Demo — Analysis Index

Repo analyzed: `contract-testing-business-demo` (TaxMesh Form 1065 microservices)

Mode: Explain-only / read-only (no installs, builds, or test runs)

Created: `docs-analysis/` (analysis artifacts only)

How to read these files

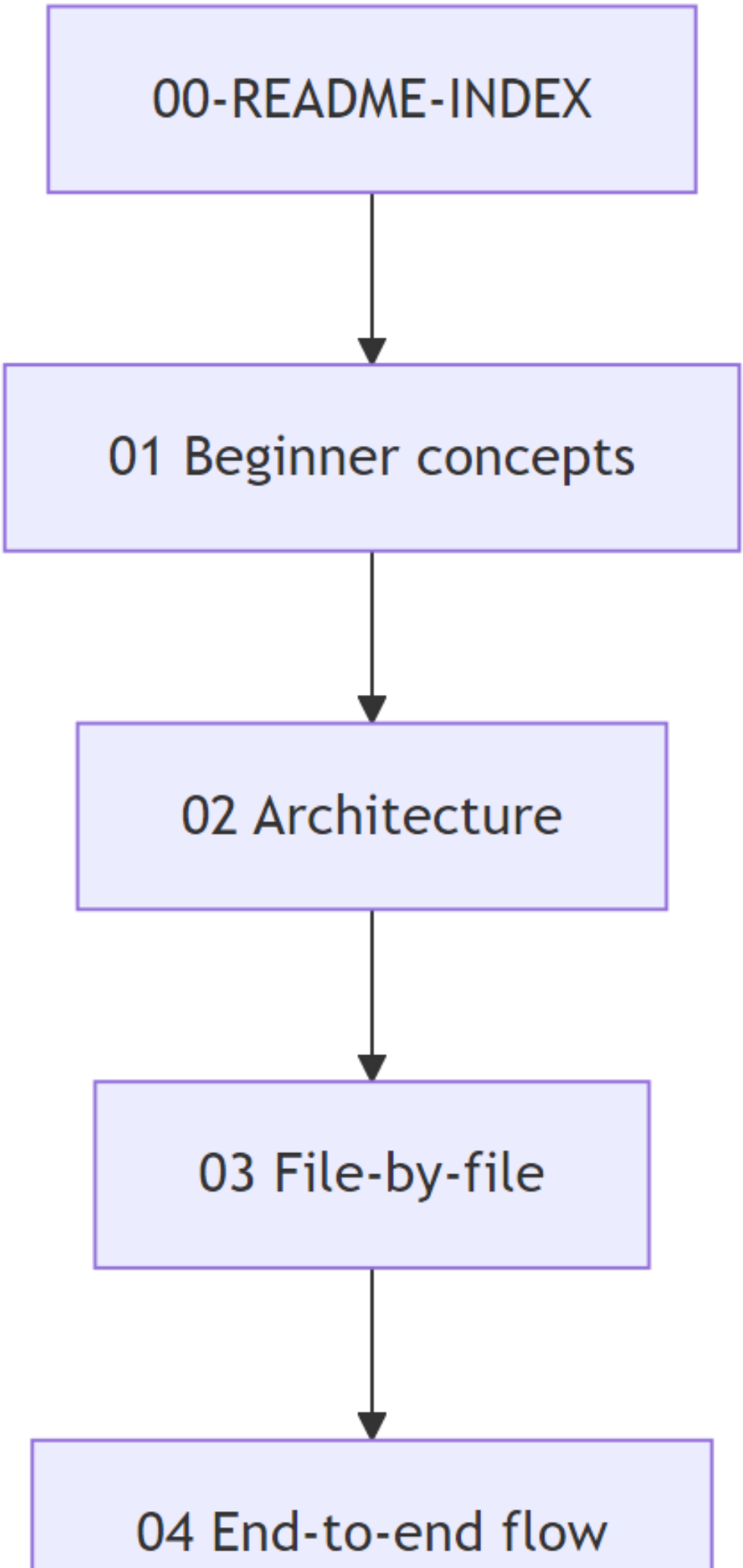

00-README-INDEX
01 Beginner concepts
02 Architecture
03 File-by-file
04 End-to-end flow

Figure 1: How to read these analysis files

Order	File	Read when you want…
0	section “Index and Discovery”	Map of the pack + Phase 1 discovery table
1	section “Beginner Concepts”	What contract testing is **in this repo**
2	section “Architecture”	Services, deps, diagrams
3	section “File-by-File Guide”	Every important file + deep dives
4	section “End-to-End Flow”	Lifecycle, happy/fail paths, sample CT
5	section “How It Runs (Explained)”	Commands explained (**not executed**)
6	section “Glossary and Q&A”	Terms, interview Qs, 2-min pitch

Tip: Prefer tables and Mermaid diagrams; use prose only as captions.

One-line verdict

Question	Answer
What is this?	Node/Express **TaxMesh** demo: 6 microservices + gateway for Form **1065**
Contract tool?	**Custom** shape checker — **not** Pact / Spring Cloud Contract
How verified?	`npm run test:contracts` → `contracts/run-contracts.js` HTTP GETs producers
Broker / CI gate?	**Not found in repo**

Phase 1 discovery table (full)

Item	Found	Evidence (file path)
Language	JavaScript (Node.js)	`package.json`, `**/*.js`
Package manager	npm	`package.json`, `package-lock.json`
Runtime (container)	Node 20 Alpine	`Dockerfile`
Web framework	Express ^4.21.2	`package.json`
Contract testing tool	Custom runner	`contracts/run-contracts.js`
Pact library	**Not found**	no dependency / import
Spring Cloud Contract	**Not found**	no Java/Maven
Pact Broker	**Not found**	—
can-i-deploy	**Not found**	—
Jest / Mocha / JUnit	**Not found**	only `test:contracts`
CI (GitHub Actions, Jenkins)	**Not found**	no workflow files
Docker	Yes	`Dockerfile`, `docker-compose.yml`
Frontend	Vanilla SPA	`frontend/`
OpenAPI / Swagger	Yes	`swagger-ui-express`, `shared/openapi.js`
Demo data	Excel sets	`testdata/set-01` … `set-05`
Domain docs	Yes	`README.md`, `docs/ARCHITECTURE.md`, `CASE_STUDY.md`

Repo tree + purpose

Path	One-line purpose
services/	Microservice entrypoints (CLM…DOC + alias folders)
gateway/	Port 4100 — UI host, proxy, filing, Swagger
frontend/	Browser SPA
shared/	Contracts, payloads, service factory, filing engine
contracts/	Contract verification runner
testdata/	Demo Excel uploads
docs/	Human architecture/API manuals
postman/	API collection
scripts/	start-all + generators
docker-compose.yml	Multi-container run
docs-analysis/	**This** teaching pack

Phase gate checklist

Gate	Topic	Status
1	Discovery / tech stack	Complete (table above)
2	Beginner concepts mapped to repo	See section “Beginner Concepts”
3	Architecture boxes → real files	See section “Architecture”
4	Important files documented	See section “File-by-File Guide”
5	Flows match code/test names	See section “End-to-End Flow”
6	Commands explained, not executed	See section “How It Runs (Explained)”
7	Glossary + uncertainties	See section “Glossary and Q&A”

Related upstream docs (already in repo)

File	Use
README.md	Quick start
docs/ARCHITECTURE.md	Official architecture
docs/API.md	API reference
CASE_STUDY.md	Case-study brief

01 — Beginner Concepts

Source file: 01-beginner-concepts.md

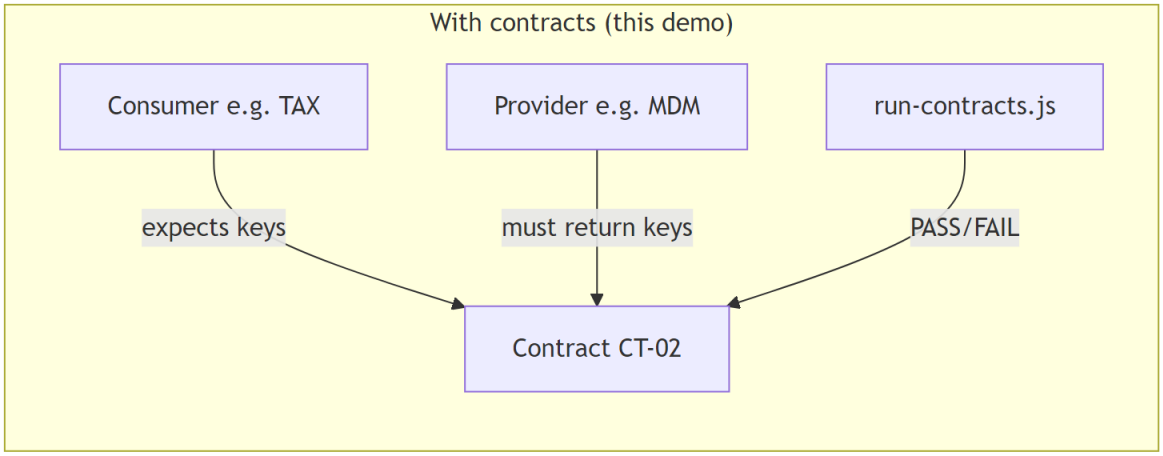
01 — Beginner Concepts (mapped to THIS repo)

Evidence paths are relative to `contract-testing-business-demo/`.

---

1. What is contract testing?

Idea: Two services agree on a **message shape** (fields + meaning). Tests check that shape **without** spinning up the full real stack every time.



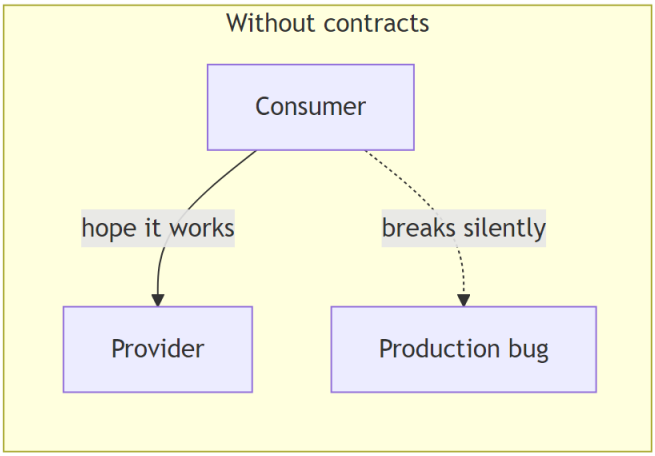


Figure 2: With vs without contracts

Question	Answer in THIS repo
What is the “contract”?	JSON **payload shape** for CT-00…CT-17 in `shared/payloads.js` + metadata in `shared/contracts.js`
How is it checked?	`contracts/run-contracts.js` → `check(ct)` fetches producer URL, asserts `contractId` + `requiredKeys`
Is it Pact / Spring Cloud Contract?	**No** — custom Node runner (Not found: `pact`, broker, can-i-deploy)

2. Why it exists (vs Integration vs E2E)

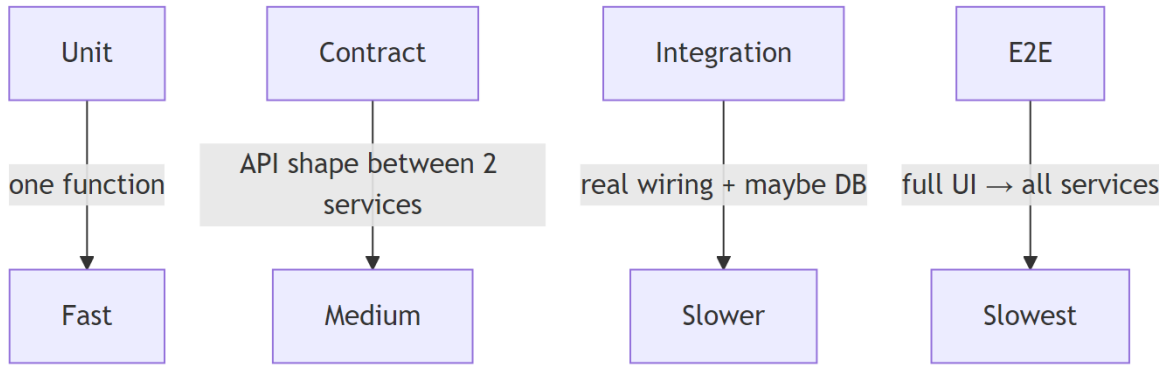


Figure 3: Test pyramid speed layers

Type	What it proves	Speed	Cost when it fails	In THIS repo?
Unit	One function works	Fast	Cheap	**Not found** (no Jest/Mocha)
Contract	Producer JSON has expected keys / `contractId`	Fast-ish	Cheap	**Yes** — `npm run test:contracts` → `contracts/run-contracts.js`
Integration	Services + shared store/filing talk correctly	Medium	Medium	**Partial** — filing APIs / uploads (`gateway/filingRoutes.js`, `shared/filingEngine.js`) act as demo integration, not a separate test suite
E2E	User journey in UI to e-file	Slow	Expensive	**UI demo** (`frontend/`) — not an automated E2E suite

3. Core terms — only what THIS repo uses

Term	Simple meaning	Used here?	Evidence
Consumer	Service that **reads** another's data	Yes	`CONTRACTS[].consumer` in `shared/contracts.js`
Producer / Provider	Service that **exposes** the JSON	Yes	`CONTRACTS[].producer`; routes in `shared/createService.js`
Contract / Pact file	Agreement on shape	Yes (as JS objects, not `.json` pact files)	`shared/payloads.js` `PAYLOADS`, `shared/contracts.js` `CONTRACTS`
Broker	Central store for contracts	**Not found in repo**	—
Provider verification	Provider proves it still matches consumer contracts	**Partial / custom**	Runner hits live producer HTTP endpoints (`run-contracts.js` `check`)
Mocks / Stubs	Fake responses	**In-memory samples**	Producers return `PAYLOADS[ct.id]` from `createService.js`
can-i-deploy	Deploy safety check via broker	**Not found in repo**	—
Downstream	Forward along journey	Yes	`direction: 'downstream'`
Upstream / reverse	Feedback ask back to earlier service	Yes	`direction: 'reverse'` ≡ API `upstream` (`apiDirection` in `contracts.js`)



Figure 4: Downstream and upstream example

4. Comparison table (memorize this)

	Contract (TaxMesh)	Integration	E2E	Unit
Scope	One CT payload shape	Several modules + filing store	Full Form 1065 UI path	Single function
Needs all 6 services up?	Yes for `test:contracts` (HTTP fetch)	Often yes	Yes + browser	No
Fails when	Missing key / wrong `contractId` / HTTP error	Wrong business flow	UI or any layer broken	Logic bug
Command (if any)	`npm run test:contracts`	Manual via UI/API	Manual UI	Not found
File	`contracts/run-contracts.js`	`shared/filingEngine.js`	`frontend/app.js`	—

5. Business problem THIS demo solves

Domain: US partnership **Form 1065** tax filing across microservices (TaxMesh).

Workspace	Code	Port	Job	Evidence
Client Management	CLM	4101	Engagement / project / job	`SERVICES.CLM` in `shared/contracts.js`
Master Data	MDM	4102	Entity, partners, ownership	`SERVICES.MDM`
Accounting	ACC	4103	Trial balance / books	`SERVICES.ACC`
Tax Computation	TAX	4104	Federal/state calc	`SERVICES.TAX`
Allocations	ALC	4105	Partner K-1 amounts	`SERVICES.ALC`
Forms & Documents	DOC	4106	1065 / K-1 package	`SERVICES.DOC`
Gateway + UI	—	4100	Orchestration + SPA	`gateway/index.js`, `frontend/`

Problem in plain English:

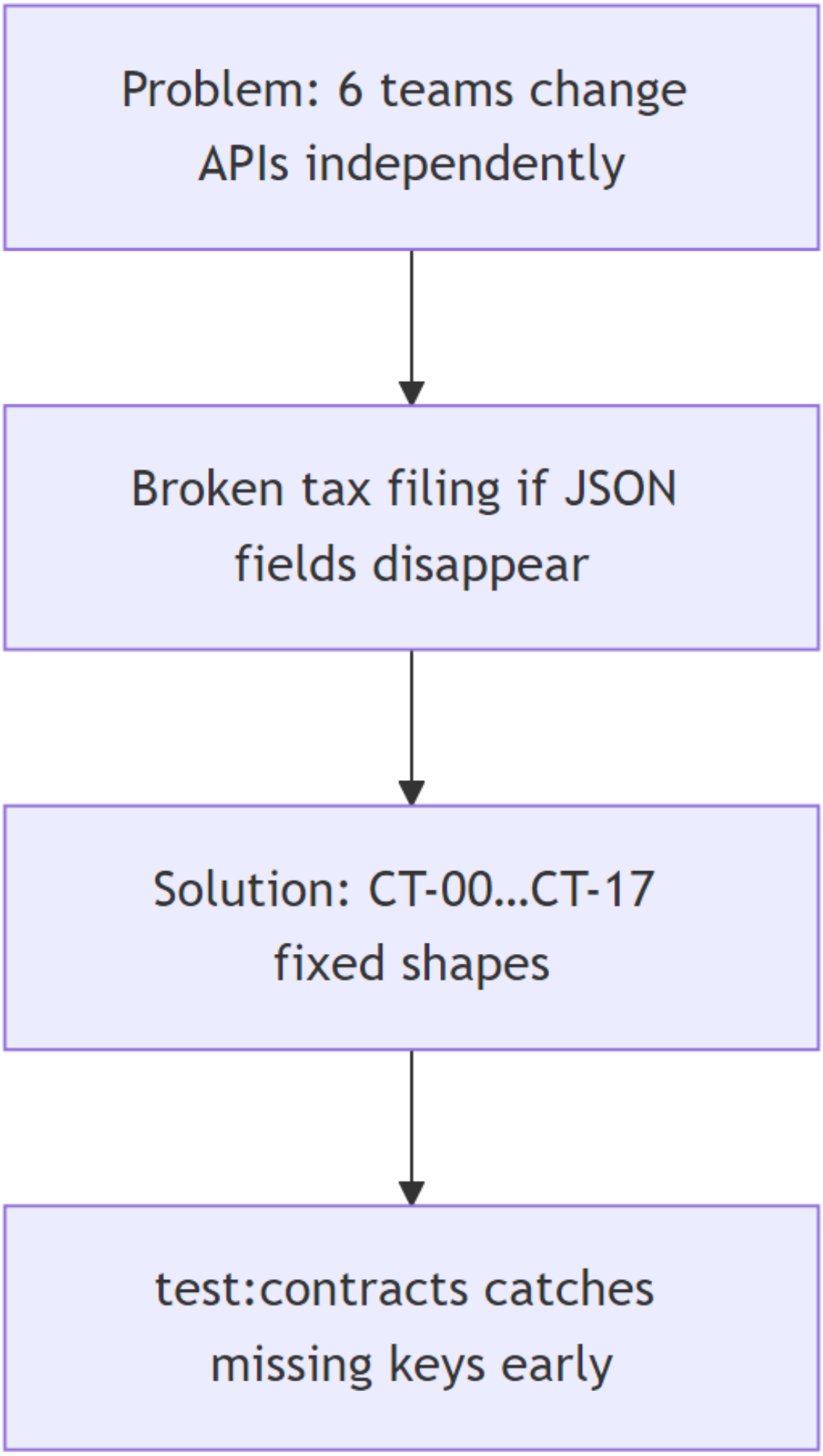

Problem: 6 teams change APIs independently
Broken tax filing if JSON fields disappear
Solution: CT-00…CT-17 fixed shapes
test:contracts catches missing keys early

Figure 5: Business problem this demo solves

Pain	How demo addresses it	File
MDM changes entity fields → TAX breaks	CT-02 shape fixed in `PAYLOADS['CT-02']`	`shared/payloads.js`
TAX needs MDM correction	Upstream CT-08	`shared/contracts.js`
Need demo data without real banks	Excel sets	`testdata/set-0N/*.xlsx`
Need one UI to drive journey	Gateway SPA	`frontend/`, `/api/filing/*`

Journey (from `README.md` / `CASE_STUDY.md`):

CLM → MDM → ACC → TAX → ALC → DOC / MeF

---

6. Mini glossary of THIS project's "contract"

Piece	Role	File
Catalog row	Who produces/consumes, path, direction	`CONTRACTS` in `shared/contracts.js`
Sample body	The agreed JSON fields	`PAYLOADS` in `shared/payloads.js`
HTTP produce	`GET` path returns sample + metadata	`createService.js` loop over `produced`
HTTP consume	Consumer pulls producer	`GET /consume/:contractId` in `createService.js`
Verification	Assert keys exist	`requiredKeys` + `check` in `run-contracts.js`

Gate 2

Check	Status
Concepts mapped to this repo (not only theory)	Done
Broker / Pact / can-i-deploy	Correctly marked **Not found**
Business domain = Form 1065 TaxMesh	Done
Uncertain	Alias folders (`services/cem` vs `services` name codes) — both valid entry points

02 — Architecture

Source file: 02-architecture.md

02 — Architecture

Every box maps to a real folder/file under `contract-testing-business-demo/`.

---

1. System context (C4-style)

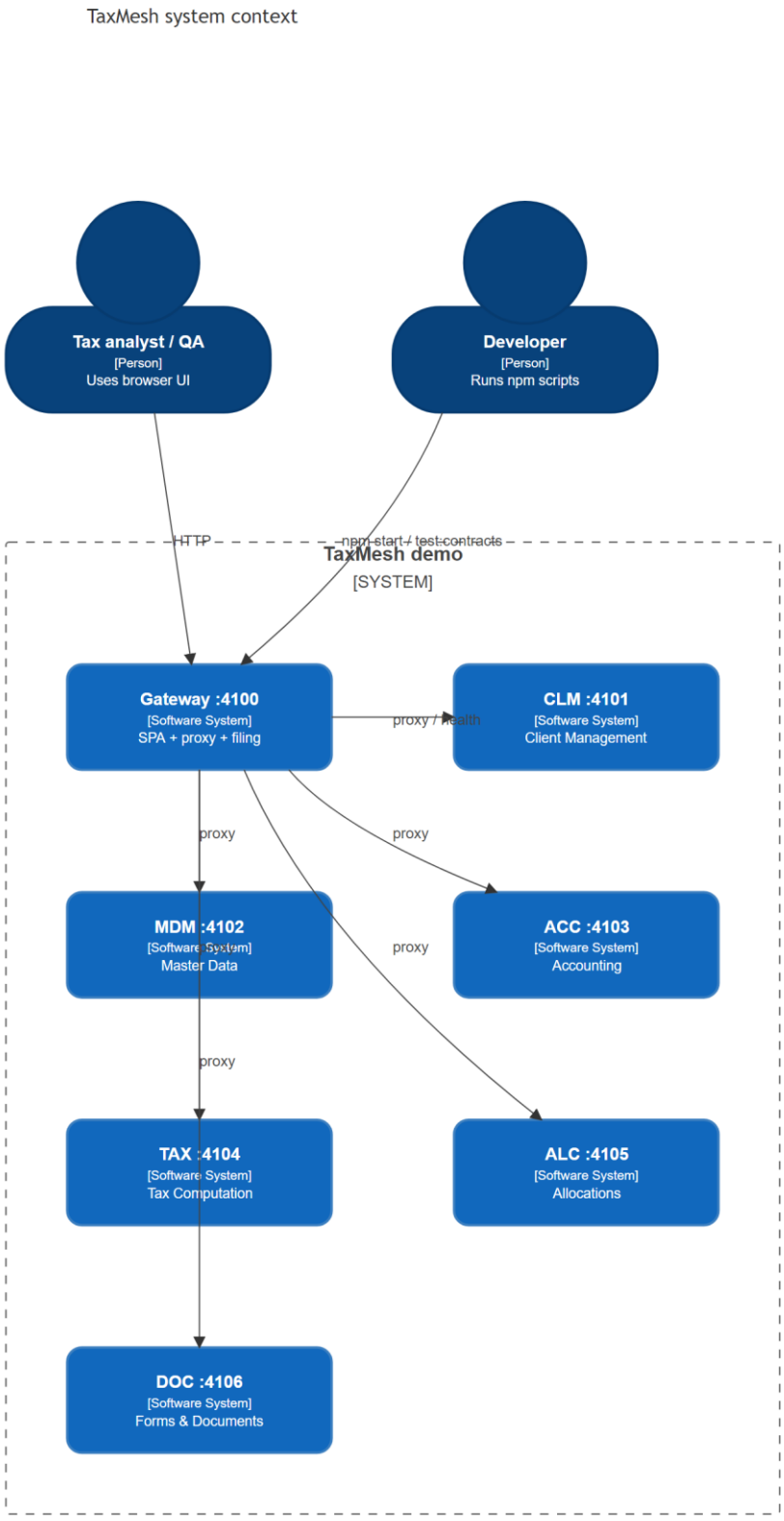


Figure 6: System context (C4-style)

Box	Real path
Gateway	`gateway/index.js`, `gateway/filingRoutes.js`
CLM	`services/cem/index.js` → `createService('CLM')`
MDM	`services/mad/index.js` → `createService('MDM')`
ACC	`services/cam/index.js` → `createService('ACC')`
TAX	`services/tim/index.js` → `createService('TAX')`
ALC	`services/rpam/index.js` → `createService('ALC')`
DOC	`services/forms-print/index.js` → `createService('DOC')`
SPA	`frontend/index.html`, `frontend/app.js`, `frontend/styles.css`
Shared catalog	`shared/contracts.js`, `shared/payloads.js`
Contract runner	`contracts/run-contracts.js`

Simpler flowchart (same as `docs/ARCHITECTURE.md`):

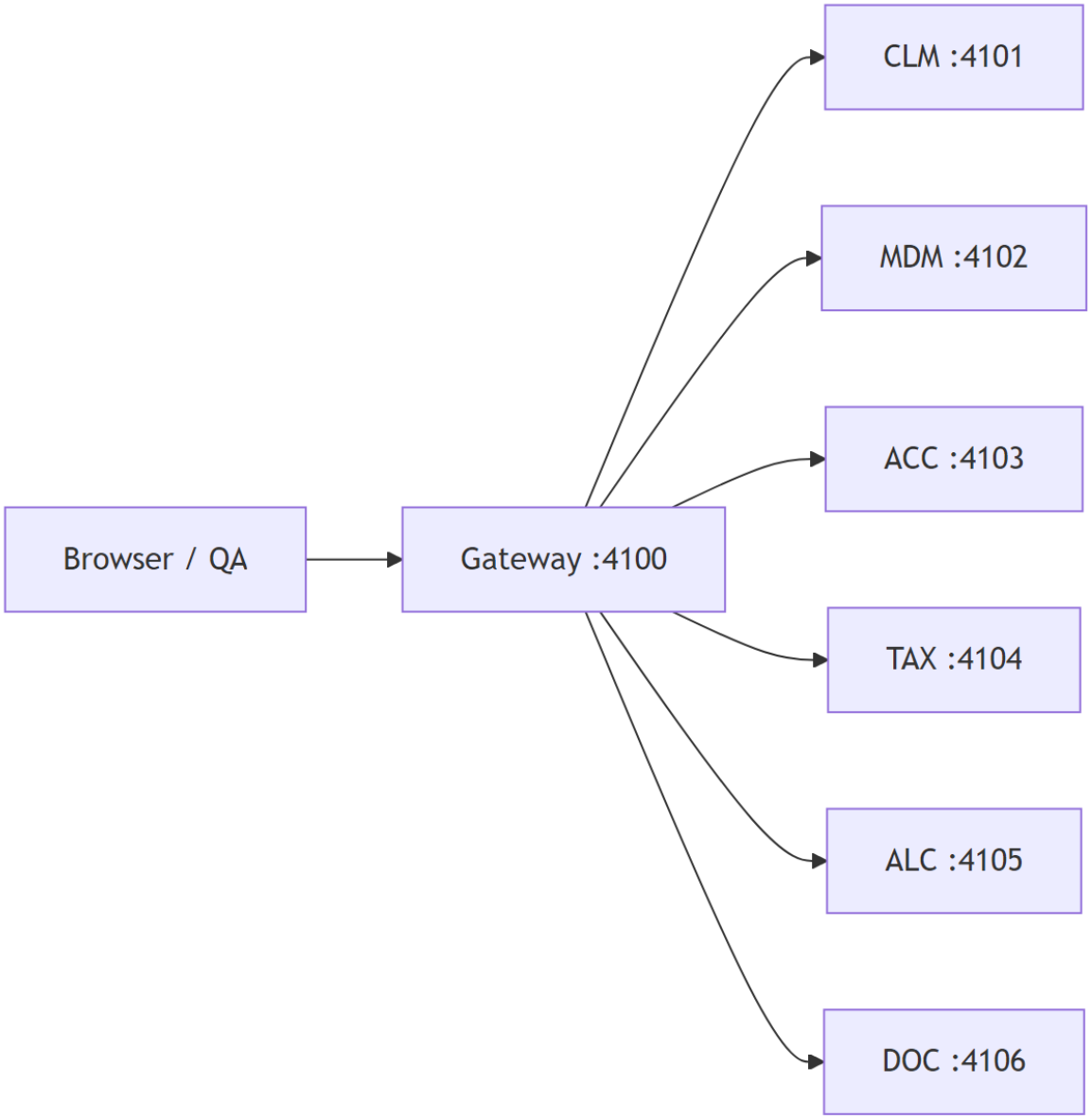


Figure 7: Gateway to six microservices

2. Component diagram

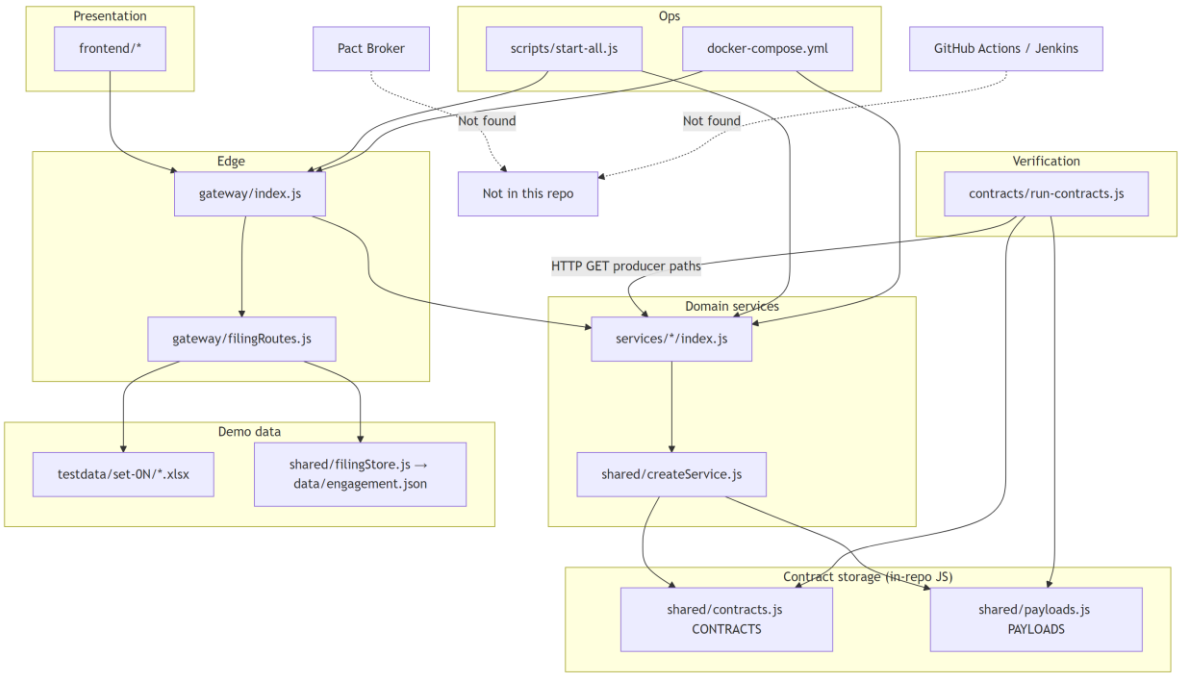


Figure 8: Component diagram

Component	Technology	Folder/File
Consumers / Producers	Express microservices	services/ + shared/createService.js
Contract catalog	JS arrays/objects	shared/contracts.js
Sample payloads	JS objects	shared/payloads.js
Contract storage	**In source code** (not broker)	same files
Broker	**Not found**	—
CI gate	**Not found**	—
Verification runner	Node `fetch` script	contracts/run-contracts.js

3. Tech stack table

Layer	Technology	Version	Purpose	Evidence
Runtime	Node.js	**20** (Docker); local version Not found pinned in README	Run services	Dockerfile FROM node:20-alpine
Package manager	npm	lockfile present	Install deps	package-lock.json
HTTP API	Express	^4.21.2	Microservices + gateway	package.json
CORS	cors	^2.8.5	Cross-origin	package.json
Uploads	multer	^2.5.0	Excel upload	gateway/filingRoutes.js
Excel	xlsx	^0.18.5	Parse/generate sheets	shared/excelImport.js
OpenAPI UI	swagger-ui-express	^5.0.1	Swagger at /api-docs	gateway/index.js
ZIP	archiver	^5.3.2	Postman/testdata zips	gateway/index.js
DOCX gen	docx	^9.14.0	Guide generation	scripts/generate-guide-docx.js
Frontend	Vanilla HTML/CSS/JS	—	SPA	frontend/
Containers	Docker Compose	compose file v-style	Multi-service	docker-compose.yml

Layer	Technology	Version	Purpose	Evidence
Contract tool	Custom runner	1.0.0 app	Shape checks	contracts/run-contracts.js

4. Dependency table (module → depends on → why)

Module	Depends on	Why	Evidence
services/*/index.js	shared/createService.js	Build Express app	each service file
shared/createService.js	contracts.js, payloads.js, flowApis.js, serviceUploads.js, serviceValidation.js, serviceV2Apis.js	Routes + payloads	requires at top
shared/flowApis.js	contracts.js, payloads.js	Downstream/upstream APIs	requires
contracts/run-contracts.js	shared/contracts.js, shared/payloads.js	Verify shapes	requires
gateway/index.js	contracts.js, payloads.js, openapi.js, filingRoutes.js, flowApis.js	Proxy + docs + UI	requires
gateway/filingRoutes.js	filingEngine.js, filingStore.js, contracts.js	Filing orchestration	requires
shared/filingEngine.js	filingStore.js, payloads.js, excelImport.js, dataFlow.js, businessValidation.js	Form 1065 journey	requires
frontend/app.js	Gateway HTTP APIs	UI calls /api/*	fetch usage in file
scripts/start-all.js	child processes of services + gateway	Boot everything	spawn list
Docker services	same Node images	Isolated ports	docker-compose.yml

5. Data / contract flow

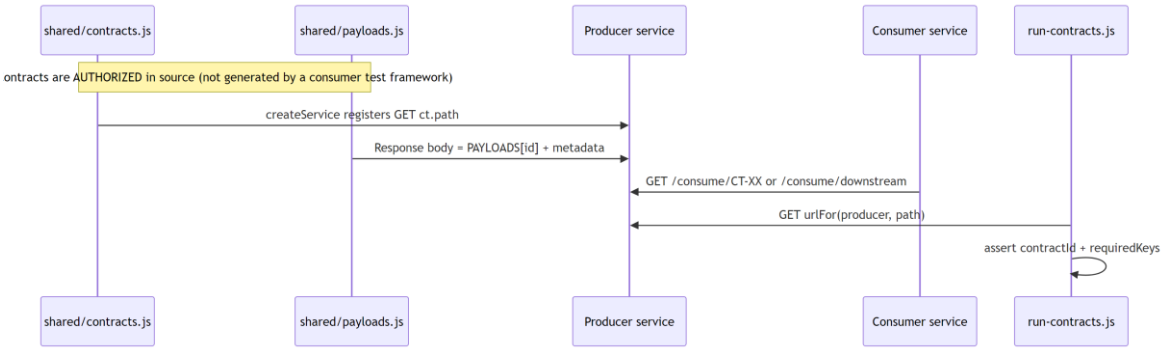


Figure 9: Contract data flow sequence

Step	Where	File
Contract **defined**	JS catalog	shared/contracts.js CONTRACTS
Shape **defined**	Sample payloads	shared/payloads.js PAYLOADS
Contract **"generated"**	Not generated at runtime from consumer tests — **hand-authored**	Not found: Pact generate step
Stored	In git as JS	same files
Served	Producer HTTP	createService.js app.get(ct.path, …)
Verified	Runner	contracts/run-contracts.js check
Published to broker	**Not found**	—

Contract mesh (downstream solid / upstream dashed)

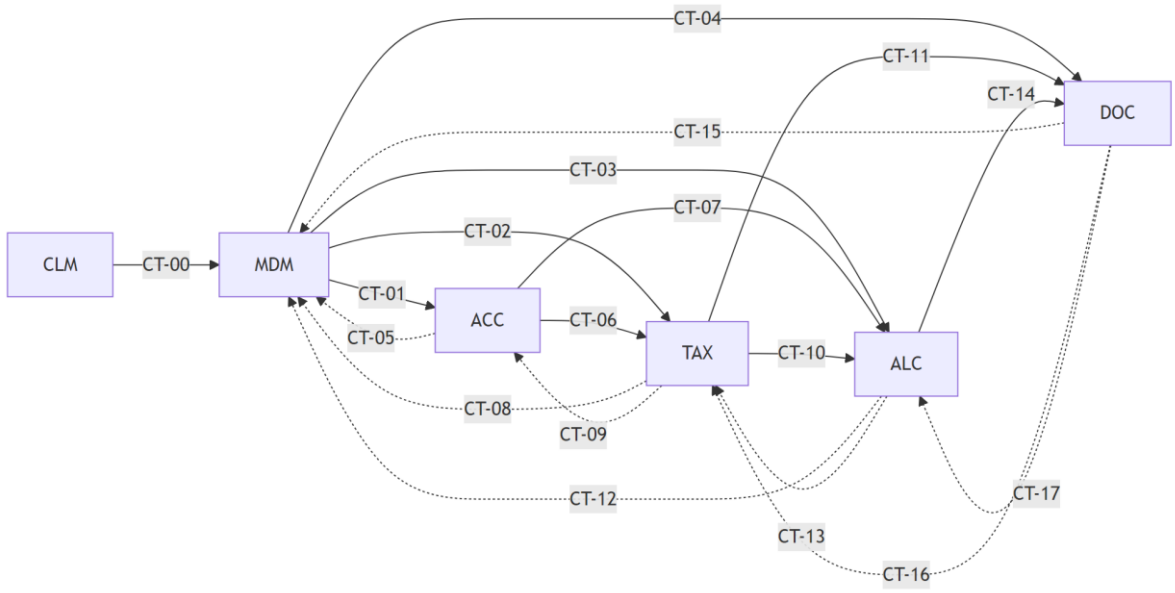


Figure 10: Contract mesh CT-00 to CT-17

Evidence: rows in `shared/contracts.js` lines ~96–122; diagram also in `docs/ARCHITECTURE.md`.

Upload-driven filing flow (separate from CT runner)

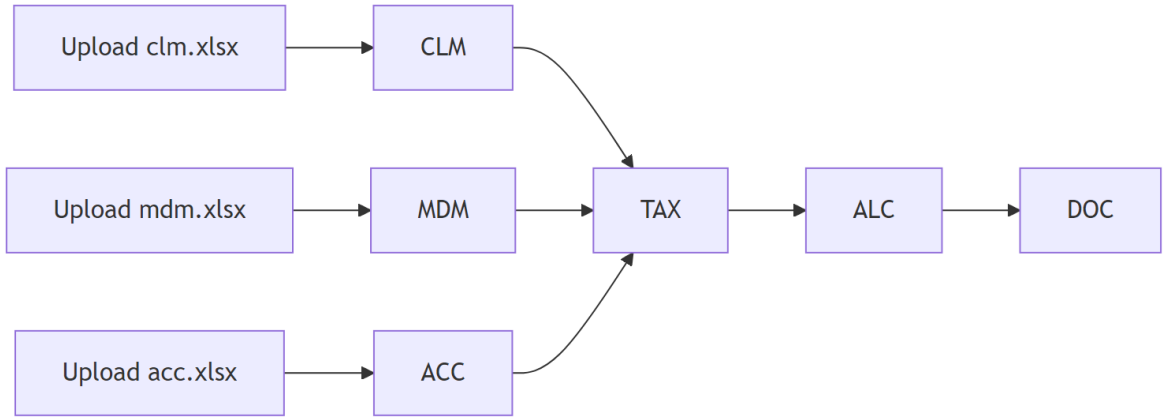


Figure 11: Upload-driven filing flow

Evidence: `shared/dataFlow.js` header comments; `README.md` “Data flow”.

6. Class / module diagram (logical)

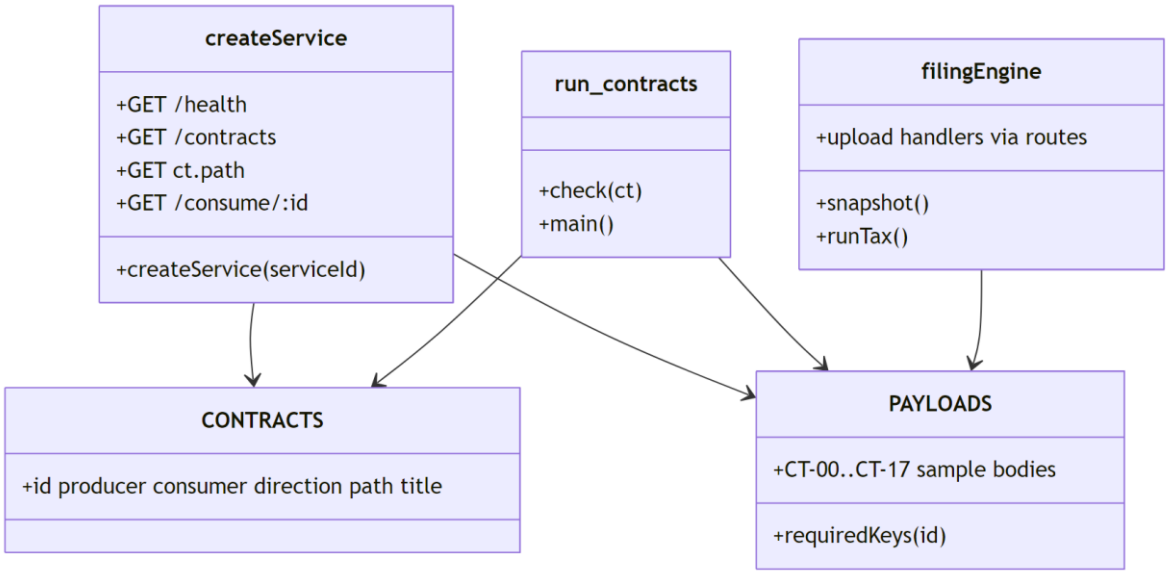


Figure 12: Module class diagram

Gate 3

Check	Status
Every diagram box → real folder/file	Done
Broker / CI	Marked Not found
Contract storage = in-repo JS	Confirmed
Uncertain	Exact local Node version outside Docker (README does not pin)

03 — File-by-File Guide

Source file: 03-file-by-file-guide.md

03 — File-by-File Guide

*Priority: **Must understand** first. Paths relative to `contract-testing-business-demo/`.*

Priority legend

Tag	Meaning
Must understand	Core to contract testing + architecture
Should know	Filing demo / ops
Can ignore at first	Generators, patches, binaries, Word/PDF case studies

---

1. Master inventory table

Root / config

File Path	Type	What it does	Key classes/methods/config	Who uses it	Related files	Priority
`package.json`	npm config	Scripts + deps	`start`, `test:contracts`, `docker:*`	Humans, Docker	`package-lock.json`	**Must**
`package-lock.json`	lockfile	Pins dependency versions	npm lock	`npm install`	`package.json`	Can ignore at first
`README.md`	docs	Quick start, ports, v2 notes	URLs, commands	Humans	`docs/*`	**Must**
`CASE_STUDY.md`	docs	Business case brief	Journey table	Humans	case study PDFs	Should know
`.gitignore`	git	Ignores `node_modules`, `data/`, `.env`	patterns	git	—	Can ignore at first
`Dockerfile`	Docker	Image: Node 20, `npm start`	`EXPOSE 4100-4106`	`docker compose`	`docker-compose.yml`	Should know
`docker-compose.yml`	Docker	6 services + gateway	`cem`…`forms-print`, `gateway`	Ops	`Dockerfile`	Should know
`cloudflared.exe`	binary	Tunnel helper (Windows)	—	Optional public URL	README `PUBLIC_BASE_URL`	Can ignore at first
`*.docx` / `*.pdf` (root)	case study assets	Source case studies	—	Reading	`CASE_STUDY.md`	Can ignore at first

Contracts

File Path	Type	What it does	Key symbols	Who uses it	Related	Priority
`contracts/run-contracts.js`	test runner	Verifies all CT payloads over HTTP	`check`, `main`	`npm run test:contracts`	`shared/contracts.js`, `shared/payloads.js`	**Must**
`shared/contracts.js`	catalog	Services, CT-00…17, URL helpers	`SERVICES`, `CONTRACTS`, `urlFor`, `flowSummary`	Almost everything	`payloads.js`	**Must**
`shared/payloads.js`	contract shapes	Sample JSON + `requiredKeys`	`PAYLOADS`, `requiredKeys`	producers + runner	`contracts.js`	**Must**

Shared runtime

File Path	Type	What it does	Key symbols	Who uses it	Related	Priority
shared/createService.js	factory	Builds each microservice Express app	createService	all services/*/index.js	flowApis, uploads, v2	**Must**
shared/flowApis.js	routes	Downstream/upstream APIs	attachFlowApis, attachPipeline, produceEnvelope	createService, gateway	contracts.js	**Must**
shared/openapi.js	OpenAPI builder	Builds Swagger document	buildOpenApi	gateway, generate:openapi	contracts	Should know
shared/serviceV2Apis.js	routes	CLM/MDM /v2/*	attachV2Apis, attachClmV2	createService	filingStore	Should know
shared/serviceUploads.js	routes	Per-service Excel upload	attachExcelUpload	CLM/MDM/ACC	excelImport	Should know
shared/serviceValidation.js	routes	TAX/ALC validate/calc	attachBusinessValidationApis	TAX/ALC	businessValidation	Should know
shared/excelImport.js	parser	Parse CLM/MDM/ACC xlsx	parseCem, parseMad, parseCam	filingEngine	xlsx	Should know
shared/filingEngine.js	domain engine	Form 1065 step machine	snapshot, runTax, uploads	filingRoutes	filingStore, dataFlow	**Must** (for demo)
shared/filingStore.js	persistence	engagement JSON + steps/gates	STEPS, read/write	filingEngine, v2	data/engagement.json	Should know
shared/dataFlow.js	graph helpers	Upload readiness + flow nodes	hasClm, buildDataFlow	filingEngine, UI APIs	—	Should know
shared/businessValidation.js	rules	TAX/ALC Form 1065 checks	validateTim, validateRpam	filingEngine, serviceValidation	—	Should know

Gateway / frontend

File Path	Type	What it does	Key symbols	Who uses it	Related	Priority
gateway/index.js	Express app	Proxy, Swagger, health, static UI	/api/health, /api/flow, /api/proxy	npm start	filingRoutes	**Must**
gateway/filingRoutes.js	router	/api/filing/* uploads & calc	createFilingRouter	gateway	filingEngine	**Must**
frontend/index.html	UI shell	Layout / nav	—	browser	app.js	Should know
frontend/app.js	UI logic	Module views, uploads, status	MODULES, showView	browser	gateway APIs	Should know
frontend/styles.css	styles	Soft teal theme	CSS	index.html	—	Can ignore at first

Services (entry points)

File Path	Type	What it does	Key	Who uses it	Related	Priority
services/cem/index.js	entry	Starts **CLM** on 4101	createService('CLM')	start-all, docker	createService	**Must**
services/mad/index.js	entry	Starts **MDM** on 4102	createService('MDM')	start-all, docker	—	**Must**
services/cam/index.js	entry	Starts **ACC** on 4103	createService('ACC')	start-all, docker	—	**Must**
services/tim/index.js	entry	Starts **TAX** on 4104	createService('TAX')	start-all, docker	—	**Must**
services/rpam/index.js	entry	Starts **ALC** on 4105	createService('ALC')	start-all, docker	—	**Must**
services/forms-print/index.js	entry	Starts **DOC** on 4106	createService('DOC')	start-all, docker	—	**Must**

File Path	Type	What it does	Key	Who uses it	Related	Priority
services/accounting/index.js	alias entry	Also ACC	createService('ACC')	optional	cam	Can ignore at first
services/master-data/index.js	alias entry	Also MDM	createService('MDM')	optional	mad	Can ignore at first
services/tax-engine/index.js	alias entry	Also TAX	createService('TAX')	optional	tim	Can ignore at first
services/allocations/index.js	alias entry	Also ALC	createService('ALC')	optional	rpam	Can ignore at first

Scripts

File Path	Type	What it does	Key	Who uses it	Related	Priority
scripts/start-all.js	launcher	Spawns 6 services + gateway	procs array	npm start	services/*	**Must**
scripts/generate-excel-datasets.js	generator	Builds testdata xlsx	—	npm run generate:excel	testdata	Can ignore at first
scripts/generate-postman.js	generator	Postman JSON	—	generate:postman	postman/	Can ignore at first
scripts/generate-guide-docx.js	generator	Word guide	—	generate:guide	docs	Can ignore at first
scripts/extract-docx.js	util	Extract docx text	—	maintainers	—	Can ignore at first
scripts/patch-*.js	one-off patches	Historical code patches	—	maintainers	—	Can ignore at first
scripts/rewrite-authors.js / strip-coauthors.sh	git helpers	Author rewrite	—	maintainers	—	Can ignore at first

Testdata / Postman / Docs

File Path	Type	What it does	Key	Who uses it	Related	Priority
testdata/index.json	catalog	Lists set-01…05	datasets	UI / gateway	xlsx files	Should know
testdata/set-0N/{clm,mdm,acc}.xlsx	demo data	Upload inputs	Excel sheets	filing upload	excelImport	Should know
postman/TaxMesh.postman_*.json	Postman	API collection	—	Postman app	generate-postman	Can ignore at first
docs/ARCHITECTURE.md	docs	Arch overview	mermaid	humans	this analysis	Should know
docs/API.md	docs	API reference	tables	humans	openapi	Should know
docs/APPLICATION_MANUAL.md	docs	Install/UI walkthrough	—	humans	README	Can ignore at first
docs/openapi.json	OpenAPI	Static export	—	Swagger consumers	openapi.js	Can ignore at first
docs/TaxMesh_API_Architecture_and_Developer_Guide.md	long guide	Full developer guide	—	humans	—	Can ignore at first

---

2. Deep dive — 8 most important files (block by block)

A. `shared/contracts.js` — Must understand

Block	What it does	Why it exists

Block	What it does	Why it exists
Header comment	Names 6 services + aliases	Orients readers to CLM…DOC
`SERVICES` object	id, name, port, role, out/in counts	Single source of truth for ports
`SERVICE_ALIASES`	CEM→CLM, A→MDM, etc.	Backward compatibility
`resolveServiceId`	Normalize any alias to current id	Safe lookups
`CONTRACTS` array	CT-00…CT-17 rows	**The contract catalog**
`urlFor`	Build `http://127.0.0.1:port` + path (or env `URL_*`)	Runner + consume calls
`contractsProducedBy` / `ConsumedBy`	Filter catalog	Wire routes per service
`normalizeDirection` / `apiDirection`	reverse ≡ upstream	API naming
`flowSummary` / `allFlowEdges`	Graph for `/api/flow`	Visualization
`module.exports`	Public API	Imported everywhere

B. `shared/payloads.js` — Must understand

Block	What it does	Why
`engagementId` / `entityId` constants	Shared demo IDs	Consistent samples
`PAYLOADS['CT-00']` … `['CT-17']`	Exact JSON fields	**This IS the contract shape**
`requiredKeys(contractId)`	All keys except `contractId`	Runner assertions
exports	Used by producers + tests	Shared truth

C. `contracts/run-contracts.js` — Must understand

Block	What it does	Why
requires	Load catalog + payloads	Need expected shapes
`check(ct)`	`fetch` producer → assert HTTP ok, `contractId`, each required key, PAYLOADS exists	Lightweight provider verification
`main()`	Loop all CONTRACTS, log PASS/FAIL, `process.exit(1)` if any fail	CLI test entry
catch on main	Unexpected errors → exit 1	Fail hard

D. `shared/createService.js` — Must understand

Block	What it does	Why
Express + cors + json	Standard HTTP service	Each MS is HTTP
`/health`	Liveness + produced/consumed lists	Gateway health aggregation
`/contracts`	Catalog for this service	Discovery
`attachFlowApis`	Downstream/upstream routes	Flow APIs
Loop `produced` → `GET ct.path`	Return `PAYLOADS` + metadata	**Producer side of contract**
`/consume/:contractId`	Pull from producer via `urlFor`	**Consumer side**
`attachPipeline`	Reachability of producers	Diagnostics
Conditional uploads / validation / v2	Feature flags by service id	CLM/MDM/ACC/TAX/ALC specifics

E. `scripts/start-all.js` — Must understand

Block	What it does	Why
`procs` list	Maps label → legacy folder script	Boot order of 7 processes

Block	What it does	Why
`spawn` loop	Child Node processes, inherit stdio	One command starts mesh
`shutdown` on SIGINT/TERM	Kill children	Clean stop

F. `gateway/index.js` — Must understand

Block	What it does	Why
Express setup + trust proxy	Tunnel-friendly	Cloudflare
`/api/filing` router	Filing journey	Demo business flow
OpenAPI + Swagger	Interactive docs	Teaching / try APIs
Static `/docs`, downloads	Serve guides/postman/testdata	Convenience
`/api/health`	Probe all 6 services	Ops dashboard
`/api/architecture`, `/api/contracts`, `/api/flow`	Mesh metadata	Teaching APIs
Proxy routes (later in file)	Forward to microservices	SPA single origin
Static frontend	Serve SPA	UI

G. `gateway/filingRoutes.js` — Should know / Must for demo

Block	What it does	Why
multer memory storage	Accept `.xlsx` ≤5MB	Upload safety
GET `/` snapshot	Current engagement state	UI refresh
reset / run / run-all / process-flow	Drive steps	Demo orchestration
validate tax/alc + calculate/allocate	Business gates	AQG-style checks
upload clm/mdm/acc	Import Excel into store	Real demo path

H. `shared/filingEngine.js` — Should know

Block	What it does	Why
`snapshot`	Full UI state + dataFlow	Single response for SPA
`isStepDone`	Step completion rules	Progress UI
jobs / quality gates	Async + AQG1–5	Case study alignment
tax/allocate/forms/efile logic	Compute Form 1065 outputs	Business demo (beyond pure CT)

---

3. Quick “read order” for beginners

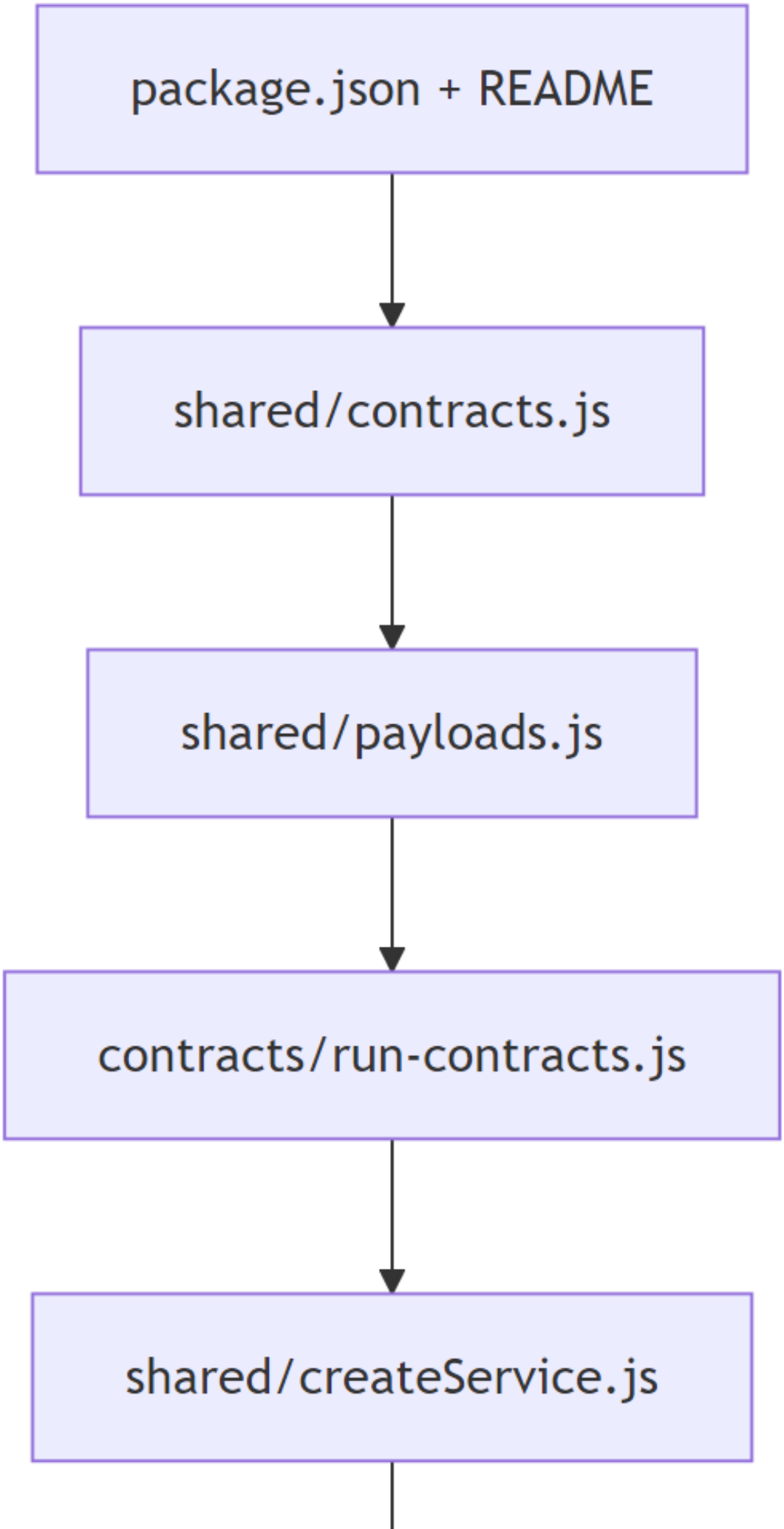

package.json + README
shared/contracts.js
shared/payloads.js
contracts/run-contracts.js
shared/createService.js

Figure 13: Recommended beginner read order

Gate 4

Check	Status
Important files tabulated	Done
8 deep dives	Done
Generators/patches marked ignorable	Done
Uncertain	Full line-by-line of entire `filingEngine.js` (~900+ lines) summarized by responsibility, not every helper

04 — End-to-End Flow

Source file: 04-end-to-end-flow.md

04 — End-to-End Flow

Theoretical walkthrough only. ***No tests were executed*** *for this analysis.*

---

1. Lifecycle sequence (how contracts work HERE)

Important difference from classic Pact: In this repo, contracts are **hand-authored in JS**, not generated by a consumer unit test framework. The "consumer test" step is replaced by **catalog + sample payloads**. Verification = HTTP GET against live producers.

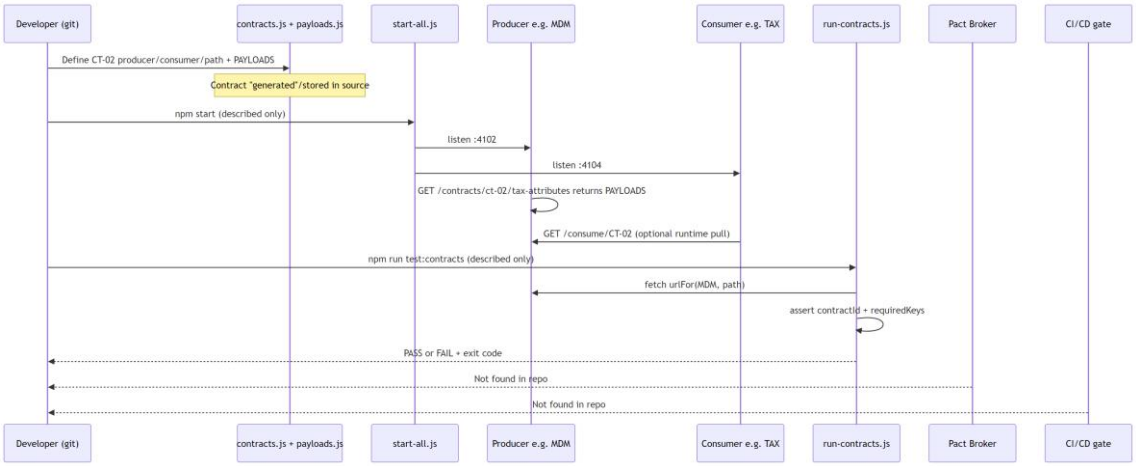


Figure 14: Contract lifecycle sequence

---

2. Numbered lifecycle table

Step	Actor	Action	File / Class / Method	Output
1	Developer	Author CT metadata	`CONTRACTS` in `shared/contracts.js`	Catalog row (id, producer, consumer, path, direction)
2	Developer	Author JSON shape	`PAYLOADS` in `shared/payloads.js`	Sample body keys
3	Service factory	Register produce route	`createService` loop `app.get(ct.path, …)`	Live endpoint
4	Ops / QA	Start mesh	`scripts/start-all.js` → `npm start`	Processes on 4100–4106
5	Consumer (runtime)	Optional pull	`GET /consume/:contractId` in `createService.js`	`{ payload, ok, sourceUrl }`
6	Contract runner	Verify all CTs	`check(ct)` in `contracts/run-contracts.js`	Console `PASS` / `FAIL`
7	Runner	Aggregate	`main()`	`N/18 contracts passed`; exit 1 if fail
8	Broker publish	—	**Not found in repo**	—
9	CI can-i-deploy	—	**Not found in repo**	—

---

3. Happy path (contracts stay valid)

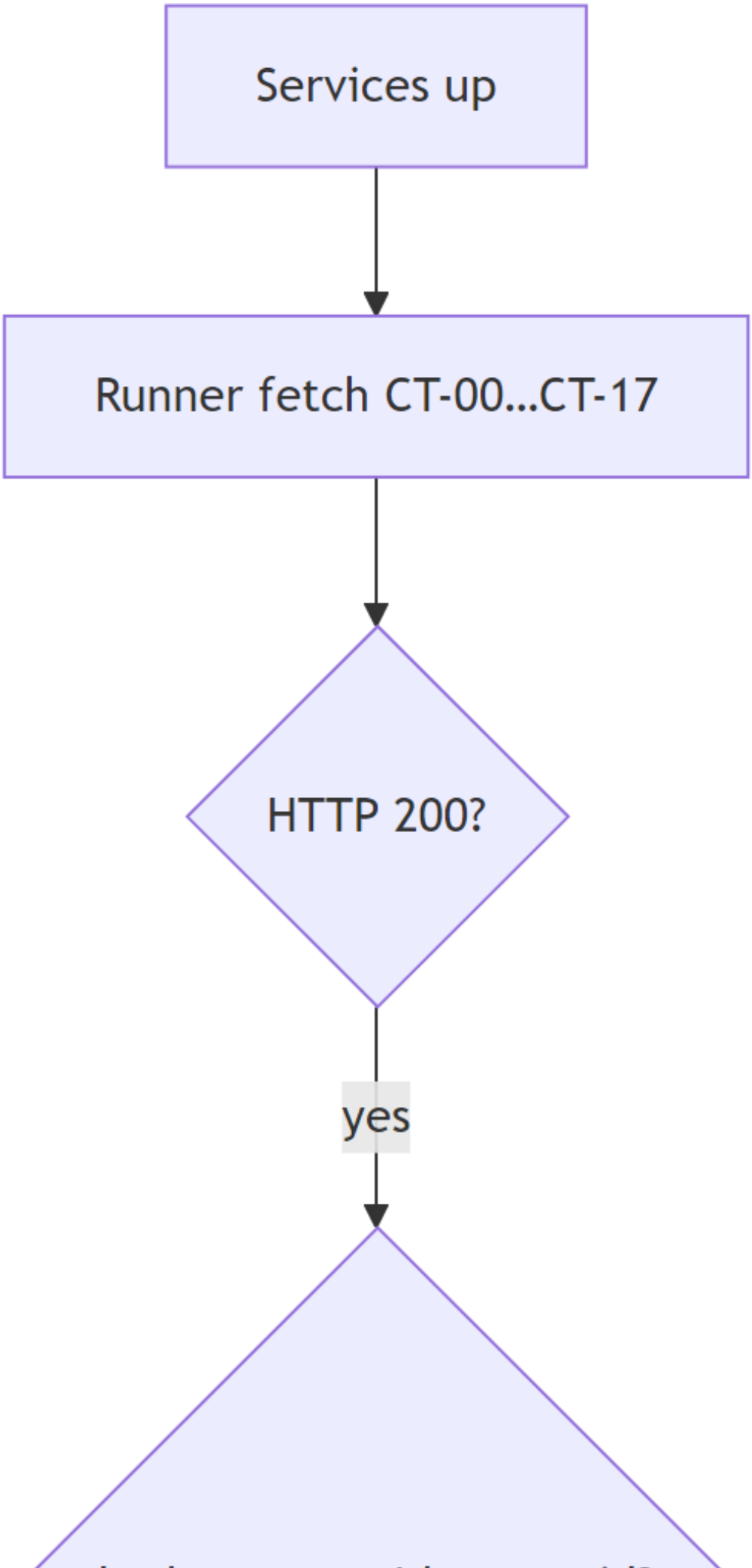

Services up
Runner fetch CT-00...CT-17
HTTP 200?
yes

Figure 15: Happy path verification

Assertion in check(ct)	Source
res.ok	HTTP status from producer
body.contractId === ct.id	Response must echo catalog id
key in body for each requiredKeys(ct.id)	Keys from PAYLOADS minus contractId
PAYLOADS[ct.id] exists	Catalog/payloads stay in sync

Expected console shape (**expected behaviour, not captured output**):

```
PASS  CT-00  CLM→MDM  (downstream)
PASS  CT-01  MDM→ACC  (downstream)
…
18/18 contracts passed
```

---

4. Contract broken path (theoretical)

Scenario: Provider MDM changes CT-02 response and **removes** field `taxAttributes` (or stops returning `contractId: 'CT-02'`).

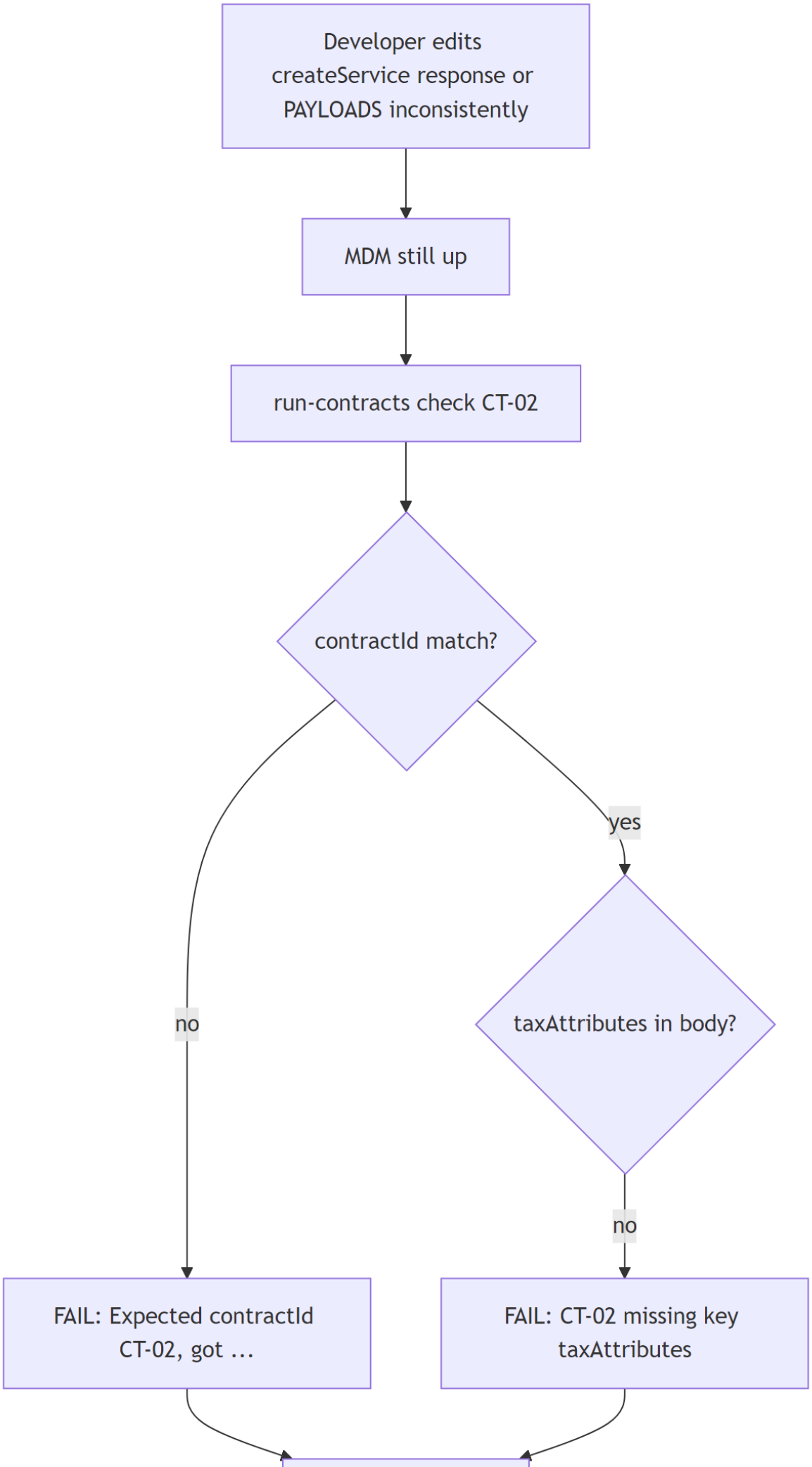

Developer edits createService response or PAYLOADS inconsistently
MDM still up
run-contracts check CT-02
contractId match?
no
yes
taxAttributes in body?
no
FAIL: Expected contractId CT-02, got ...
FAIL: CT-02 missing key taxAttributes

Figure 16: Contract broken path

If change is…	Which check fails	Why (code)
Producer down / wrong port	HTTP ${status} or fetch throw	check first lines
Wrong contractId in JSON	Expected contractId…	body.contractId !== ct.id
Missing field still listed in PAYLOADS	missing key "…"	requiredKeys loop
Delete sample from PAYLOADS but leave CONTRACTS	No sample payload for CT-xx	last assert in check
Change only consumer code	Runner may still **PASS**	Runner does **not** execute consumer business logic — only producer shape

Filing demo impact (separate from runner): If Excel/MDM data lacks partners/EIN, `validateTim` in `shared/businessValidation.js` adds findings like `TIM-BV-02` — UI/API returns 422 on tax calculate (`gateway/filingRoutes.js`), even if CT runner still passes sample payloads.

---

5. Sample contract explained — CT-02

Catalog row (`shared/contracts.js`):

Field	Value	Meaning
id	CT-02	Contract identifier
producer	MDM	Who serves JSON
consumer	TAX	Who needs it
direction	downstream	Forward along journey
path	/contracts/ct-02/tax-attributes	HTTP path on MDM
title	Entity tax attributes	Human label

Sample payload (`shared/payloads.js` → `PAYLOADS['CT-02']`):

Field	Example value	Meaning
contractId	CT-02	Must match catalog; asserted by runner
engagementId	ENG-2025-1065-001	Ties data to engagement
entityId	ENT-ALPHA-LP	Partnership entity
taxAttributes	object	Tax setup block
taxAttributes.filingType	1065	Form type
taxAttributes.fiscalYearEnd	12-31	FYE
taxAttributes.multiState	true	Multi-state flag
taxAttributes.states	['NY','CA']	State list
taxAttributes.section754Election	true	754 election

Extra fields added at produce time (`createService.js`):

Field	Meaning
producer	Service id
consumer	Target consumer
direction / flow	downstream vs upstream
producedAt	ISO timestamp

Required keys for runner = `engagementId`, `entityId`, `taxAttributes` (via `requiredKeys` — all keys except `contractId`).

---

6. Full contract matrix (evidence)

ID	Producer → Consumer	Direction	Path
CT-00	CLM → MDM	downstream	/contracts/ct-00/engagement-context
CT-01	MDM → ACC	downstream	/contracts/ct-01/entities
CT-02	MDM → TAX	downstream	/contracts/ct-02/tax-attributes
CT-03	MDM → ALC	downstream	/contracts/ct-03/ownership
CT-04	MDM → DOC	downstream	/contracts/ct-04/filing-meta
CT-05	ACC → MDM	reverse	/contracts/ct-05/mapping-gaps
CT-06	ACC → TAX	downstream	/contracts/ct-06/book-balances
CT-07	ACC → ALC	downstream	/contracts/ct-07/capital-map
CT-08	TAX → MDM	reverse	/contracts/ct-08/attribute-gaps
CT-09	TAX → ACC	reverse	/contracts/ct-09/btt-asks
CT-10	TAX → ALC	downstream	/contracts/ct-10/taxable-income
CT-11	TAX → DOC	downstream	/contracts/ct-11/tax-summary
CT-12	ALC → MDM	reverse	/contracts/ct-12/ownership-issues
CT-13	ALC → TAX	reverse	/contracts/ct-13/allocation-needs
CT-14	ALC → DOC	downstream	/contracts/ct-14/partner-allocations
CT-15	DOC → MDM	reverse	/contracts/ct-15/entity-print-needs
CT-16	DOC → TAX	reverse	/contracts/ct-16/tax-line-needs
CT-17	DOC → ALC	reverse	/contracts/ct-17/capital-restatement

Source: `shared/contracts.js` `CONTRACTS` array.

---

7. Filing happy path (business demo, not the CT runner)

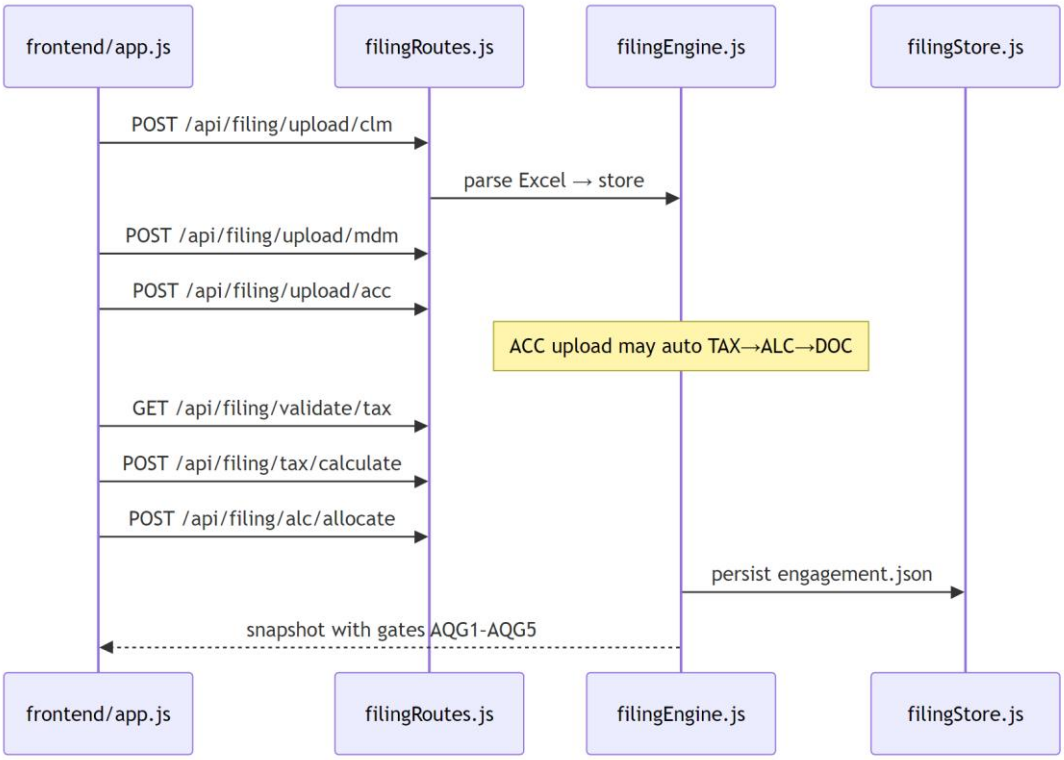


Figure 17: Filing happy path sequence

Gate 5

Check	Status
Flows match actual test names (`check`, `main`, CT ids)	Done
Happy + broken paths from code	Done
Sample CT-02 field table	Done
Broker/CI steps	Not found — shown as such
Uncertain	Exact console formatting if Node version differs — based on `console.log` template strings only

05 — How It Runs (Explained)

Source file: 05-how-it-runs-explained.md

05 — How It Runs (Explained Only — NOT Executed)

> *Every command row is marked **Not executed - explained only**. Analysis is from README + `package.json` + source.*

---

1. Prerequisites

Tool / item	Version found	Required for	Evidence
Node.js	**20** in Docker image; local exact version **Not found** in README	Run app & scripts	Dockerfile `FROM node:20-alpine`
npm	Comes with Node; lockfile present	`npm install`, scripts	`package-lock.json`
Docker + Compose	Compose file present	Optional container run	`docker-compose.yml`
Git	Repo has `.git`	Clone only	folder `.git`
Pact CLI / Broker	**Not found**	—	—
Java / Maven / Gradle	**Not found**	—	—
Browser	Any modern	UI at :4100	README.md
Optional `PUBLIC_BASE_URL`	env var	Tunnel/Swagger absolute host	README.md, gateway/index.js

Install step described in README (not run here):

```
npm install
```

---

2. Commands table

Command	What it does internally	What it produces	Where output appears	Status
`npm install`	Reads `package.json` / lockfile; installs Express, multer, xlsx, etc.	`node_modules/`	Console + folder	**Not executed - explained only**
`npm start`	Runs `node scripts/start-all.js` → spawns 6 services + gateway	7 Node processes	Console logs per service; ports 4100–4106	**Not executed - explained only**
`npm run start:clm` (etc.)	Starts one service entry	One listener	Console `[CLM] …`	**Not executed - explained only**
`npm run start:gateway`	`node gateway/index.js`	Gateway only	:4100	**Not executed - explained only**
`npm run test:contracts`	`node contracts/run-contracts.js` → `check` each CT	PASS/FAIL lines; exit 0/1	stdout	**Not executed - explained only**
`npm run generate:excel`	`scripts/generate-excel-datasets.js`	`testdata/set-0N/{clm,mdm,acc}.xlsx`	filesystem	**Not executed - explained only**
`npm run generate:openapi`	Inline node write of `buildOpenApi()`	`docs/openapi.json`	filesystem + console	**Not executed - explained only**
`npm run generate:postman`	`scripts/generate-postman.js`	Postman JSON files	`postman/`	**Not executed - explained only**
`npm run generate:guide`	`scripts/generate-guide-docx.js`	Word guide	`docs/*.docx`	**Not executed - explained only**
`npm run docker:up`	`docker compose up --build`	Containers for cem…gateway	Docker + mapped ports	**Not executed - explained only**

Command	What it does internally	What it produces	Where output appears	Status
npm run docker:down	docker compose down	Stops containers	Docker	**Not executed - explained only**

What `npm start` boots (`scripts/start-all.js`)

Label	Script path	Port (from SERVICES)
CLM	services/cem/index.js	4101
MDM	services/mad/index.js	4102
ACC	services/cam/index.js	4103
TAX	services/tim/index.js	4104
ALC	services/rpam/index.js	4105
DOC	services/forms-print/index.js	4106
Gateway	gateway/index.js	4100

What `test:contracts` does inside `check(ct)`

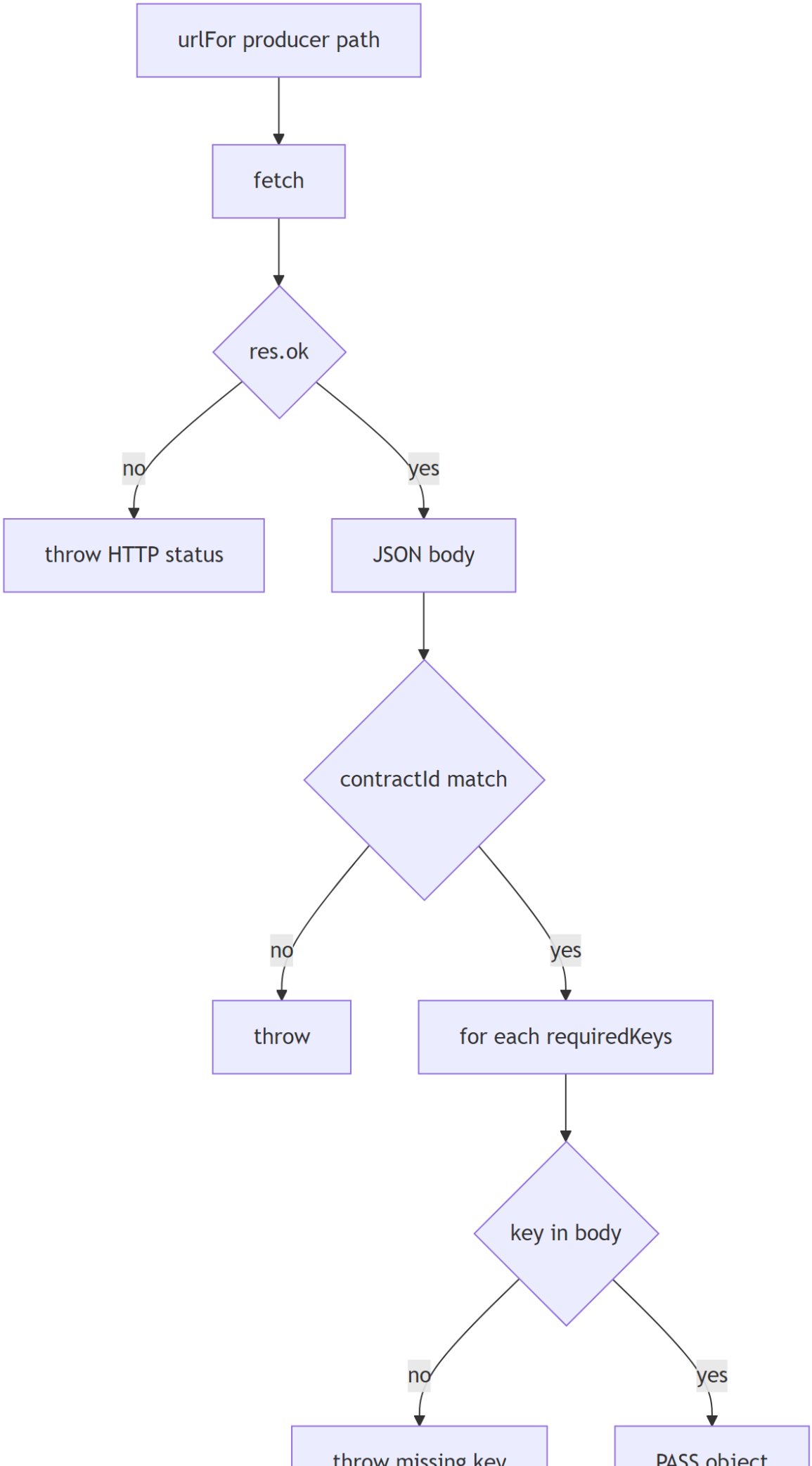

urlFor producer path
fetch
res.ok
no
yes
throw HTTP status
JSON body
contractId match
no
yes
throw
for each requiredKeys
key in body
no
yes
throw missing key
PASS object

Figure 18: test:contracts check flow

3. Expected PASS vs FAIL (not real output)

PASS — expected behaviour

Signal	Meaning	Code
Line `PASS CT-xx PROD→CONS (direction)`	That contract's producer responded with correct id + keys	`console.log` in `main`
Final `18/18 contracts passed`	All `CONTRACTS.length` ok	end of `main`
Process exit code **0**	`failed === 0`	no `process.exit(1)`

FAIL — expected behaviour

Signal	Meaning	Code
`FAIL CT-xx … HTTP 404 from …`	Producer route missing / service down	thrown Error
`FAIL … Expected contractId CT-02, got …`	Wrong id in body	assert
`FAIL … CT-02 missing key "taxAttributes"`	Shape drift	`requiredKeys` loop
Final `k/18 contracts passed` with k<18	Partial success	filter PASS count
Exit code **1**	Any failure	`if (failed) process.exit(1)`

Filing API FAIL (separate)

Signal	Meaning	Evidence
HTTP **422** on `/api/filing/validate/tax`	Business validation failed	`filingRoutes.js` + `validateTim`
Finding codes `TIM-BV-01`…	Missing engagement/entity/TB	`businessValidation.js`

4. Useful URLs (when running — described from README)

URL	Resource
http://127.0.0.1:4100	TaxMesh UI
http://127.0.0.1:4100/api-docs	Swagger
http://127.0.0.1:4100/openapi.json	OpenAPI JSON
http://127.0.0.1:4100/api/flow	Downstream + upstream catalog
http://127.0.0.1:4100/download/postman.zip	Postman ZIP

5. Troubleshooting

Problem	Likely cause	Fix (from repo config)
`test:contracts` all FAIL / fetch errors	Services not started	Run `npm start` first (README order)
One service unhealthy in `/api/health`	Process crashed / port busy	Check console from `start-all.js`; free ports 4101–4106
Docker services can't reach each other	Need compose DNS names	Env `URL_CEM…` in `docker-compose.yml` `x-env`
Upload rejected	Not `.xlsx` or >5MB	`multer` filter/limits in `filingRoutes.js`
Swagger wrong host behind tunnel	Absolute localhost in older setups	Set `PUBLIC_BASE_URL` or use same-origin `/`

Problem	Likely cause	Fix (from repo config)
		(README.md, gateway)
Postman download 404	Files not generated	`npm run generate:postman` (README)
Guide docx 404	Not generated	`npm run generate:guide`
Ownership / tax validation errors	Bad Excel data	Use `testdata/set-0N`; ownership must total 100% (docs/API.md)
Legacy path confusion	CEM vs CLM	Aliases in `SERVICE_ALIASES` — both work
Looking for Pact broker	Tool not used	Use custom runner only

---

6. Docker layout (explained)

Compose service	Command	Host port	Maps to workspace
`cem`	`node services/cem/index.js`	4101	CLM
`mad`	`node services/mad/index.js`	4102	MDM
`cam`	`node services/cam/index.js`	4103	ACC
`tim`	`node services/tim/index.js`	4104	TAX
`rpam`	`node services/rpam/index.js`	4105	ALC
`forms-print`	`node services/forms-print/index.js`	4106	DOC
`gateway`	`node gateway/index.js`	4100	UI + proxy

Evidence: `docker-compose.yml`.

Gate 6

Check	Status
Commands from README/`package.json` only	Done
Nothing executed	Confirmed (explain-only)
PASS/FAIL labelled expected behaviour	Done
Uncertain	Whether `fetch` is available on very old Node without Docker — Docker pins 20; README silent on minimum local Node

06 — Glossary and Q&A

Source file: 06-glossary-and-qna.md

06 — Glossary + Q&A

---

1. Glossary

Term	Simple meaning	Where in THIS repo
TaxMesh	Name of this Form 1065 microservices demo	`README.md`, `package.json` description
Contract	Agreed JSON shape between producer & consumer	`shared/contracts.js` + `shared/payloads.js`
CT-00 … CT-17	The 18 named contracts	`CONTRACTS` array
Producer	Service that exposes the contract HTTP GET	`CONTRACTS[].producer`
Consumer	Service that needs that data	`CONTRACTS[].consumer`
Downstream	Forward along CLM→…→DOC	`direction: 'downstream'`
Upstream / reverse	Feedback ask to an earlier service	`direction: 'reverse'`; API name `upstream`
Payload	Sample JSON body	`PAYLOADS`
requiredKeys	Keys the runner requires (except contractId)	`payloads.js` `requiredKeys`
Provider verification	Checking producer still returns shape	`contracts/run-contracts.js` `check`
Pact	Popular contract-testing tool	**Not found in repo**
Broker	Central contract store	**Not found in repo**
can-i-deploy	Broker deploy safety check	**Not found in repo**
CLM	Client Management (engagement)	port 4101, `services/cem`
MDM	Master Data	port 4102, `services/mad`
ACC	Accounting	port 4103, `services/cam`
TAX	Tax Computation	port 4104, `services/tim`
ALC	Allocations	port 4105, `services/rpam`
DOC	Forms & Documents	port 4106, `services/forms-print`
CEM/MAD/CAM/TIM/RPAM/FORMS	Legacy aliases for above	`SERVICE_ALIASES`
A/B/C/D/E	Letter aliases MDM…DOC	`SERVICE_ALIASES`
Gateway	Front door :4100	`gateway/index.js`
Filing engine	Form 1065 step orchestration	`shared/filingEngine.js`
AQG1–AQG5	Quality gates on filing journey	`docs/ARCHITECTURE.md`, `filingStore` steps
MeF / e-file	Electronic filing end state	filing status `efiled`
Form 1065	US partnership tax return	domain of demo
K-1 / K-3	Partner schedule forms	DOC / ALC payloads
createService	Factory that builds each MS	`shared/createService.js`
Flow APIs	`/downstream`, `/upstream`, `/consume/*`	`shared/flowApis.js`
Swagger / OpenAPI	Interactive API docs	`/api-docs`, `shared/openapi.js`
Excel import	Upload-driven demo data	`testdata`, `excelImport.js`
Mock/stub (here)	In-memory `PAYLOADS` returned by producers	`createService` produce routes
Integration test	Multi-component behaviour	Filing demo (not a separate suite)
E2E	Full UI journey	Manual via `frontend/`

Term	Simple meaning	Where in THIS repo
urlFor	Builds service base URL + path	contracts.js
correlationId	Ties engagement work together	filing / CLM context

2. Interview-style Q&A (15) — answers from THIS project

#	Question	Short answer
1	What problem does this repo demonstrate?	Keeping API shapes stable across 6 tax microservices (Form 1065) using CT-00…CT-17.
2	Is this Pact?	No. Custom Node runner in contracts/run-contracts.js.
3	Where is the contract stored?	In source: shared/contracts.js + shared/payloads.js (no broker).
4	What does npm run test:contracts do?	HTTP GETs each producer path; asserts contractId and required keys.
5	Who is producer for CT-02?	MDM; consumer is TAX (shared/contracts.js).
6	Downstream vs upstream?	Downstream = journey forward; upstream = reverse feedback (apiDirection).
7	How does a service expose a contract?	createService registers GET ct.path returning PAYLOADS[ct.id].
8	How does a consumer pull a contract?	GET /consume/:contractId fetches producer via urlFor.
9	What happens if a required field is removed?	Runner logs FAIL … missing key and exits 1.
10	What are AQG1–AQG5?	Filing quality gates (CLM→MDM … through MeF), documented in docs/ARCHITECTURE.md.
11	Why Excel uploads?	Drive realistic Form 1065 demo data into the filing engine (testdata/).
12	What does the gateway do?	SPA, Swagger, filing APIs, proxy to services (gateway/index.js).
13	How do you start everything?	npm start → scripts/start-all.js spawns 7 processes.
14	Is there CI that blocks deploy?	**Not found in repo** (no workflows / can-i-deploy).
15	Why legacy folder names like cem?	Historical codes; aliases map CEM→CLM; Docker/start still use old folder paths.

3. "Explain this project in 2 minutes" script

*TaxMesh is a **Node/Express demo** of a **US partnership Form 1065** pipeline split into **six microservices**: CLM, MDM, ACC, TAX, ALC, and DOC, plus a **gateway UI on port 4100**.*

*Between those services there are **18 contracts, CT-00 through CT-17** — some **downstream** (forward data) and some **upstream/reverse** (feedback asks).*

*Unlike Pact, contracts here are **plain JavaScript objects** in `shared/contracts.js` and `shared/payloads.js`. Each producer serves its sample JSON on a fixed path; `npm run`*

*`test:contracts` **fetches those URLs** and checks that the **`contractId` and required fields** are still present.*

*Separately, you can **upload Excel** files and walk the **filing journey** with business validation and quality gates — that's the business demo layered on top of the contract mesh.*

*There is **no Pact broker and no CI can-i-deploy** in this repository; the teaching focus is the **mesh + shape checks + Form 1065 story**.*

---

4. Learning path

Level	Topic	Where in repo
Beginner	Ports, journey, what CT means	README.md, CASE_STUDY.md, docs-analysis/Beginner Concepts
Beginner	Read one contract end-to-end	CONTRACTS CT-00 + PAYLOADS['CT-00']
Beginner	Run story (when you execute later)	npm start, UI :4100
Intermediate	Service factory + produce/consume	shared/createService.js, shared/flowApis.js
Intermediate	Contract runner assertions	contracts/run-contracts.js
Intermediate	Downstream vs upstream catalog	GET /api/flow code in gateway/index.js
Intermediate	Architecture diagrams	docs/ARCHITECTURE.md, docs-analysis/Architecture
Advanced	Filing engine + gates	shared/filingEngine.js, businessValidation.js
Advanced	OpenAPI generation	shared/openapi.js
Advanced	Docker multi-service env URLs	docker-compose.yml URL_*
Advanced	Compare to real Pact/broker	Contrast with **Not found** pieces; design what you'd add

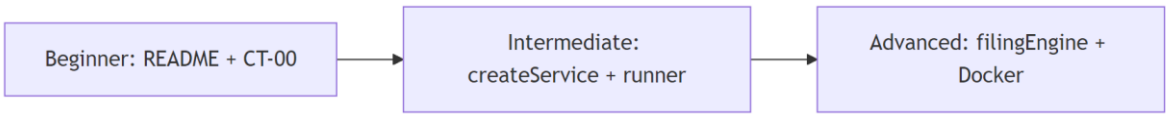


Figure 19: Learning path progression

---

5. Final summary of uncertainties

Item	Status
Pact / broker / can-i-deploy / CI	Confirmed **Not found**
Local Node version (non-Docker)	**Not pinned** in README
Alias service folders vs start-all paths	Both exist; start-all uses legacy names
Whether PAYLOADS nested objects are deeply schema-validated	**No** — runner only checks top-level keys via requiredKeys
Automated E2E/unit suites	**Not found**

Gate 7

Check	Status
Glossary of terms used	Done
15 interview Q&As mapped to project	Done

Check	Status
2-minute script	Done
Learning path	Done
Uncertainties listed	Done

Build metadata

Figures embedded: **19** / **19**. Fixes applied during diagram render: 2 — fig-08: defined sink node X for Broker/CI edges, fig-16: removed quotes inside node labels (taxAttributes).